



DLDBSOOFPP01

OBJECT ORIENTATED AND FUNCTIONAL PROGRAMMING WITH PYTHON

HABIT TRACKER : CONCEPTUAL PHASE

Name and Surname	Matriculation Number
Dylan Vosloo	9212174

June 2, 2024

1 Explanation of Concept

1.1 Aim

The old homage "roman was not built in a day" implies that in order to achieve great things, simple tasks need to be completed consistently. A definable task completed periodically can be defined as a habit. In order to grow/develop as individuals mentally, physically and spiritually, it is important to have strong habits which are completed consistently. However, lack of motivation can cause many a civilization to collapse.

In order to ensure that strong habits are carried out periodically, it is important to track the progress of such activities, thus maintaining motivation. This can be achieved through the use of a so called "habit tracker". The aim of this portfolio is to outline the process of conceptualizing such a habit tracking tool.

1.2 Desired content and goals

A habit tracker requires that a habit can be:

- Defined
- Analyzed
- Completed

A habit tracker is an extensive program that should be implemented with a suitable programming language such as Python. The IDE chosen for implementation is PyCharm with Python version 3.11.9. PyCharm provides a user friendly interface with interpreter and compiler translator abilities as well as a terminal interface for easy error identification.

In order to define a habit, a user should be able to provide a suitable name for the habit. The habit should pass a uniqueness check to ensure that multiple instances of the habit is not stored.

Habits may have different periods, for this application two periods are implemented including "Daily" and "Weekly" and upon creation a user should be define which period the habit falls into.

Further, it is important to record the start date and time of the habit in order to track whether a habit has been completed within the allocated period. In order to capture this information, the built in Python 'input' function may be used. Appropriate string handling should be implemented in order to store the information in a format that is repeatable by the user. The date and time of the habit creation can be implemented using the *datetime* Python function.

In order to add habits, the habits and the details need to be stored permanently. This can be achieved through the *sqlite3* package for database manipulation. This will allow a user to freely add new habits to their Daily/Weekly routines. It should be possible to remove a habit, in the instance where a user is no longer interested in tracking the habit information. The user should be able to enter the habit name and, after a verification process, have the habit removed from the *sqlite3* database.

Some of the most important data stored by a habit tracker is whether a habit has been completed or not, and how long a habit has been kept successfully by a user. A successful habit would be one that has been completed on-time over at least one period. This can be referred to as a streak.

To track whether a habit has been completed, the user can run the Python application and enter the name of a habit that has been completed. The *datetime* function can then be used to determine the current time and whether the habit has been completed within 24 hours (Daily) or within 7 days (Weekly) of the habits creation or last completion. The database should be updated with new last completion times and the streak for the habit should be incremented.

Other features include displaying all habits, displaying habits within a period, displaying the max streak of all habits and the streak of a current habit. These aspects are numerous and will result in a detailed Python program. The best way to achieve these outcomes would be to make use of functional and object-orientated programming techniques.

This is to be implemented by creating a Python class called ‘Habits’, this class can be used as an instance for any habit that the user wishes to add, analyze or complete. Further, it is possible to add methods that can be used to determine the longest streak for each habit and overall as well as complete/remove habits. With each method, the database should be accessed to determine the information that is requested by the user.

Since the application is designed to be used by anyone, the program should be accessible through an interface. In this case, a simple Command Line Interface (CLI) will be exposed to the user by using the Python *Click* module. This will allow the user to easily be able to access all methods within the program and to utilize the habit tracker completely. In order to provide a user-friendly interface, each method needs to be clearly named and described through the use of doc-strings and method descriptions. A write up of the program operation will be provided to the user in the form of a simple *README.txt*. Further, 4 weeks of habit tracking data will be provided to the user for testing of the basic operations of the habit tracker.

1.3 Plan of Action

In order to complete the habit tracker certain steps need to be completed.

Firstly, a *Habits.py* file can be created and a ‘Habits’ class can be defined. The *init* function can be passed all the relevant information of each habit instance including: name, start date, completion, streak.

Within the *Habits* class methods can be defined for each of functions that the user may require such as *addHabit* and *removeHabit*. A further python file such as *HabitDB.py* can be created for all of the relational database manipulation. The parameters from this file can be imported into the *Habits.py* file for complete integration of the program. Each method should be written in such a way that the operation can be tested throughout the development phase with the use of a unit test module provided with the instructions of the program.

After completion of the program the application can be compiled into an executable file using PyCharm’s built-in compiler functions. The database file along with a list of

instructions are to be provided as a zip file and uploaded into a public *Github* repository. This way a user has access to all the files required to download, install and run the habit tracker!

2 Graphical Concept Representation

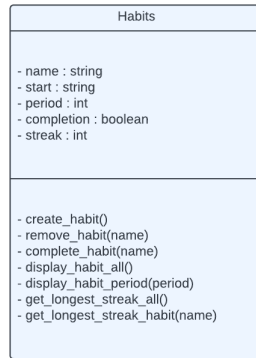


Figure 1: UML Diagram Depicting the Class Habits, with Attributes and Methods

Based on Figure 1 only 1 class is necessary for the habit tracker implementation and so no relationships exist between other classes.

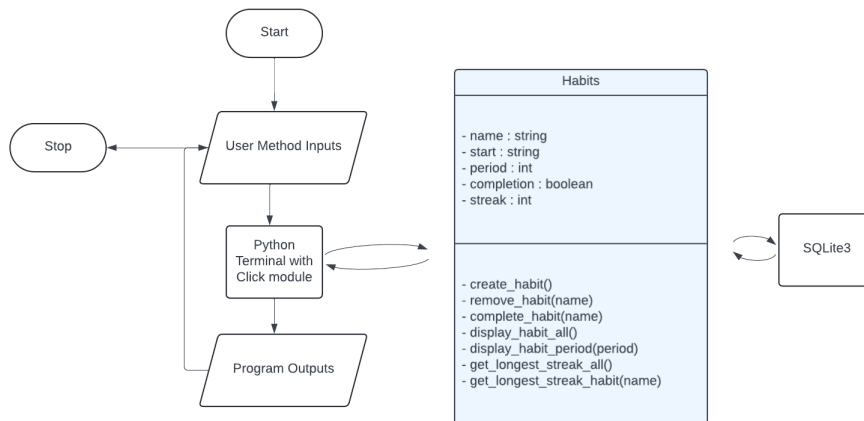


Figure 2: Flowchart of the Habit Tracker Implementation

Figure 2 depicts the flow of the Habit Tracker, where the user can input information into terminal using the Click module. This allows access to the Habits class and all methods defined in this class. The methods link to the SQLite3 relational database for permanent saving of habitual information. Each method then provides outputs to the user. The user can then continue to interact with the program or terminate the session.